

GCN: Illicit Transaction Detection on the Elliptic Bitcoin Network

Anti-money-laundering as node classification with a 2-layer Graph Convolutional Network

[Your name]

[roll no.]

Abstract

We apply the Graph Convolutional Network (GCN) of Kipf and Welling [1] to a real anti-money-laundering task: classifying the 203,769 Bitcoin transactions of the Elliptic dataset [2] as *licit* or *illicit*. The propagation rule is implemented from scratch as sparse matrix products, not with a library layer. The task is harder than citation or co-purchase benchmarks: only 9.8% of labelled transactions are illicit, the graph is split into 49 time steps and the model is tested on *future* ones, and strong tabular baselines exist. Trained on time steps 1–34 and tested on 35–49, the two-layer GCN reaches an illicit F1 of 0.560 ± 0.034 over 5 seeds (0.572 for seed 42). Compared with the same network without the graph, it is far more precise (0.79 versus 0.56) and ranks illicit transactions much better (average precision 0.60 versus 0.44), but its F1 is only slightly higher, and with local features only the graph does not help F1 at all. A Random Forest remains the strongest model (0.810), as in the original benchmark. We analyse the training behaviour, the effect of class imbalance, and over-smoothing in trained models of increasing depth. We also show that every model fails after a dark-market shutdown at time step 43, which makes concept drift the main open problem.

1 The Elliptic Bitcoin Dataset

Bitcoin is pseudonymous: every transaction is public, but the people behind it are not. This makes it attractive for ransomware, scams, darknet markets and Ponzi schemes, and it gives financial institutions an *anti-money-laundering* (AML) problem: which transactions should be flagged? The **Elliptic dataset** [2], released by the blockchain analytics company Elliptic together with MIT-IBM Watson AI Lab, is the largest public labelled transaction graph for this task. It is a graph in which

- each **node** is a Bitcoin transaction (203,769 in total);
- each **edge** is a flow of bitcoin: an output of one transaction is spent as an input of the next (234,355 directed edges);
- each node has a **feature vector** of 165 anonymised values: 93 *local* features of the transaction itself (number of inputs/outputs, fee, output volume, ...) and 72 *aggregated* features that summarise its one-hop neighbours (maxima, minima, standard deviations, correlations);
- each node carries a **time step** from 1 to 49 (roughly two weeks apart) and a **label**: illicit (4,545), licit (42,019) or unknown (157,205).

Figure 1 shows a real piece of the graph, the *Wallet* $\rightarrow$ *Tx* $\rightarrow$ *Tx* $\rightarrow$ *Tx* chain of the problem statement: about twenty illicit transactions, including the highlighted one, all send funds into the same unlabelled transaction, a typical “fan-in” pattern for collecting proceeds. Such a pattern is invisible to a model that looks at one transaction at a time, and it is exactly what a GCN can use.

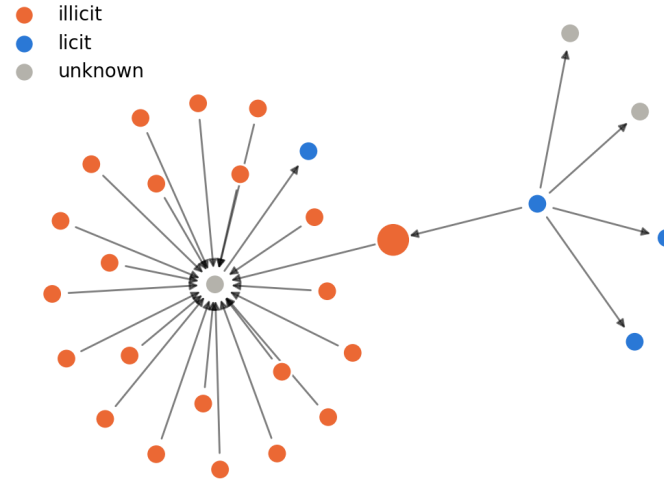
2-hop neighbourhood of illicit transaction 68954616 ($t = 35$)

Figure 1: Two-hop neighbourhood of an illicit test transaction (large node). Arrows follow the flow of bitcoin.

Three properties set Elliptic apart from Cora or Amazon Photo (Table 1):

- **Imbalance.** Only 9.8% of labelled nodes are illicit, and 77% of nodes have no label at all. On the test period a model that always answers “licit” is 93.5% accurate yet catches no criminal, so accuracy is useless as a metric; we use the *illicit F1 score*.
- **Time.** No edge connects two time steps, so the graph is made of 49 separate transaction graphs and has exactly 49 connected components, one per time step. We follow the **temporal split** of Weber et al.: train on time steps 1–34 and test on 35–49. Test transactions therefore sit in graphs the model has never seen: the GCN must generalise *inductively* to new graphs, not only to new nodes of a known graph.
- **Sparsity and homophily.** The average degree is only 2.30 (median 2, max 473). Among edges whose two ends are labelled, 95.4% join transactions of the same class, and a labelled neighbour of an illicit transaction is illicit 54.1% of the time, against a base rate of 9.8% (and 2.4% for neighbours of licit transactions). The graph therefore carries a strong signal.

Nodes / features / classes	203,769 / 165 / 2 (+ unknown)
Edges	234,355 directed (treated as undirected for the GCN)
Labels	4,545 illicit / 42,019 licit / 157,205 unknown
Time steps / connected components	49 / 49
Train (time steps 1–34)	29,894 labelled, 3,462 illicit (11.6%)
Test (time steps 35–49)	16,670 labelled, 1,083 illicit (6.5%)
Average degree	2.30

Table 1: The Elliptic dataset as loaded in the notebook.

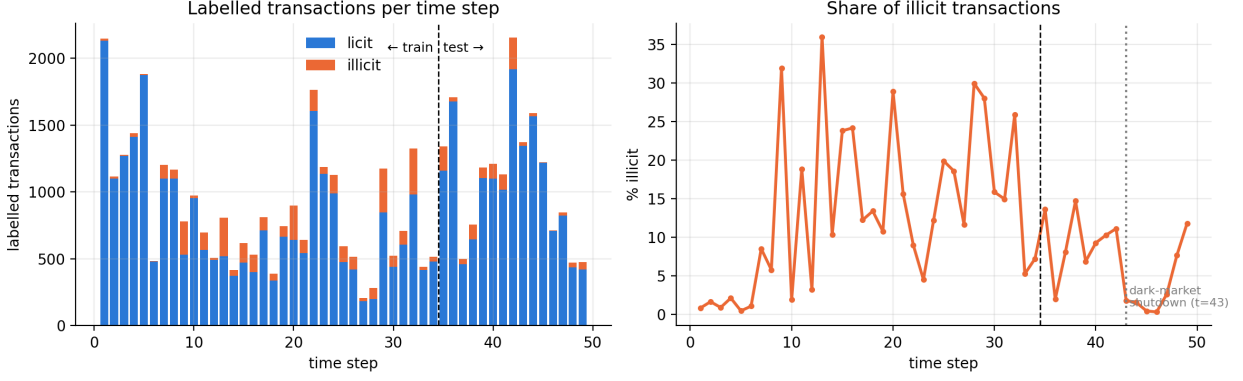


Figure 2: Labelled transactions per time step (left) and the share that is illicit (right). The dashed line is the train/test boundary; the dotted line marks the dark-market shutdown reported by Weber et al.

2 Introduction: What, Why and How of a GCN

2.1 What is a GCN?

A **Graph Convolutional Network** is a neural network that works directly on graph-structured data. An image has a fixed grid, so a convolution can slide one filter over it; a node in a graph has a variable number of unordered neighbours, so ordinary convolution does not apply. A GCN replaces the sliding filter by *neighbourhood aggregation*: in every layer, each node’s representation is recomputed from its own features and those of its neighbours, with the same weights for all nodes. After k layers a node has seen information from up to k hops away.

2.2 Why use a GCN for fraud detection?

- **Crime leaves a structural trace.** Illicit funds are split, merged and relayed through chains of transactions. Whether a transaction is suspicious depends on where its money came from and where it goes, not only on its own amount or fee.
- **Few labels, many nodes.** Only 23% of transactions are labelled. The unlabelled ones still pass messages, so a label near them informs their neighbours.
- **Efficiency.** A layer touches only existing edges, so its cost grows with the number of edges; the Elliptic graph with about 670 thousand non-zeros is processed in well under a second on a laptop CPU.
- **No node ordering.** Relabelling the transactions only permutes the rows of the output.

2.3 How does it work?

Let the graph have N nodes, adjacency matrix $A \in \mathbb{R}^{N \times N}$ and feature matrix $X \in \mathbb{R}^{N \times F}$. We first add self-loops so that a node keeps its own information, then normalise by degree:

$$\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}, \quad \tilde{A} = A + I_N, \quad \tilde{D}_{ii} = \sum_j \tilde{A}_{ij}. \quad (1)$$

One GCN layer is

$$H^{(l+1)} = \sigma(\hat{A} H^{(l)} W^{(l)}), \quad H^{(0)} = X, \quad (2)$$

where $W^{(l)}$ is a weight matrix shared by all nodes and σ is a non-linearity (ReLU). The product HW transforms every node’s features in the same way, and multiplying by $\hat{A}$ aggregates them

over the neighbourhood. For a single node v ,

$$h_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_u \tilde{d}_v}} h_u^{(l)} W^{(l)} \right), \quad (3)$$

a degree-weighted average over the node and its neighbours. The symmetric weights matter on Elliptic: some transactions have hundreds of inputs or outputs (maximum degree 473), and without the $1/\sqrt{\tilde{d}_u \tilde{d}_v}$ factor such hubs would dominate their neighbours' representations.

2.4 Where the propagation rule comes from

The layer rule is a simplified *spectral* graph convolution [1]. **(1) Laplacian.** With degree matrix D , the normalised Laplacian is $L = I - D^{-1/2} A D^{-1/2}$. For a signal f on the nodes, $f^\top L f = \frac{1}{2} \sum_{i,j} A_{ij} (f_i / \sqrt{d_i} - f_j / \sqrt{d_j})^2$, so L measures how much f varies across edges. **(2) Graph Fourier basis.** L is symmetric positive semi-definite, $L = U \Lambda U^\top$; eigenvectors with small eigenvalues are smooth over the graph, those with large eigenvalues oscillate. A spectral filter is $g_\theta \star f = U g_\theta(\Lambda) U^\top f$. **(3) Polynomial filters.** Computing U costs $O(N^3)$, impossible for $N \approx 2 \times 10^5$. Approximating g_θ by a polynomial of degree K in L removes the eigendecomposition and makes the filter K -hop local, because $(L^K)_{ij} = 0$ for nodes more than K hops apart. **(4) First order.** Kipf and Welling keep $K = 1$, approximate the largest eigenvalue by 2 and tie the two coefficients, giving $g_\theta \star f \approx \theta (I + D^{-1/2} A D^{-1/2}) f$. **(5) Renormalisation trick.** $I + D^{-1/2} A D^{-1/2}$ has eigenvalues in $[0, 2]$, so stacking it makes activations explode or vanish. Replacing it by $\hat{A}$ of Eq. (1), whose eigenvalues lie in $(-1, 1]$, and generalising θ to a matrix W , gives Eq. (2).

3 Working Mechanism of Our GCN

3.1 Task

Given the transaction graph of Section 1, predict for every labelled transaction in time steps 35–49 whether it is illicit, using its 165 features, the graph structure and the labels of time steps 1–34 only.

3.2 The model

Stacking two layers of Eq. (2) with a softmax output gives

$$Z = \text{softmax} \left(\hat{A} \text{ReLU}(\hat{A} X W^{(0)} + b^{(0)}) W^{(1)} + b^{(1)} \right), \quad (4)$$

with $X \in \mathbb{R}^{203769 \times 165}$, $W^{(0)} \in \mathbb{R}^{165 \times 100}$, $W^{(1)} \in \mathbb{R}^{100 \times 2}$, and $Z_{i,1}$ the predicted probability that transaction i is illicit. We flag transaction i when $Z_{i,1} \geq 0.5$.

We also study two variants built from the same code. The **MLP** replaces $\hat{A}$ by the identity I and so sees only each transaction's own features; comparing it with the GCN isolates the contribution of the graph. The **Skip-GCN** of Weber et al. adds a linear path from the input to the output,

$$Z = \text{softmax} \left(\hat{A} \text{ReLU}(\hat{A} X W^{(0)} + b^{(0)}) W^{(1)} + b^{(1)} + X W_{\text{skip}} \right), \quad (5)$$

so that a transaction's own evidence reaches the output without being averaged with its neighbours'.

3.3 Loss for an imbalanced problem

With about one illicit transaction in ten, an unweighted cross-entropy is minimised quite well by predicting "licit" for almost everything. Following [2], we use a **weighted cross-entropy** with

$w_{\text{licit}} = 0.3$ and $w_{\text{illicit}} = 0.7$ over the labelled training set $\mathcal{Y}_L$:

$$\mathcal{L} = -\frac{1}{\sum_{i \in \mathcal{Y}_L} w_{y_i}} \sum_{i \in \mathcal{Y}_L} w_{y_i} \ln Z_{i, y_i}. \quad (6)$$

The loss uses only the 29,894 labelled training transactions, but gradients flow through $\hat{A}$, so unlabelled transactions shape the hidden representations of their labelled neighbours.

3.4 Metrics

With TP , FP , FN counted for the illicit class, $\text{precision} = TP / (TP + FP)$ is the share of flagged transactions that really are illicit (the cost of false alarms for an investigator), $\text{recall} = TP / (TP + FN)$ is the share of illicit transactions that are caught, and the **illicit F1** $= 2PR / (P + R)$ is the benchmark’s headline metric. We also report micro-F1 (equal to accuracy) and the average precision (AP, the area under the precision–recall curve), which does not depend on the 0.5 threshold.

3.5 Algorithm

Algorithm 1: Training and evaluation, as implemented in the notebook

Input: graph (A, X) , labels Y , masks $\mathcal{M}_{\text{train}}$ (t 1–34), $\mathcal{M}_{\text{test}}$ (t 35–49)

- 1 Standardise X with the mean and std of time steps 1–34; build $\hat{A}$ once (sparse);
- 2 Compute $P \leftarrow \hat{A}X$ once ; // $\hat{A}$ and X are fixed
- 3 Initialise $W^{(0)}, W^{(1)}$ (Glorot), biases $\leftarrow 0$;
- 4 **for** $\text{epoch} = 1$ **to** 200 **do**
- 5 $Z \leftarrow \hat{A} \text{ReLU}(PW^{(0)} + b^{(0)}) W^{(1)} + b^{(1)}$ over the *whole* graph;
- 6 $\mathcal{L} \leftarrow$ weighted cross-entropy (Eq. 6) on $\mathcal{M}_{\text{train}}$ only;
- 7 back-propagate and take one Adam step;
- 8 every 10 epochs: log loss and train/test illicit F1 (for plotting only);

Output: illicit precision, recall, F1 and AP on $\mathcal{M}_{\text{test}}$ after the final epoch

We have no validation period to spare (time steps are few and the test period must stay in the future), so we do *not* select epochs or hyperparameters on the test set: we fix standard values in advance and report the last epoch.

4 Model Architecture and Implementation

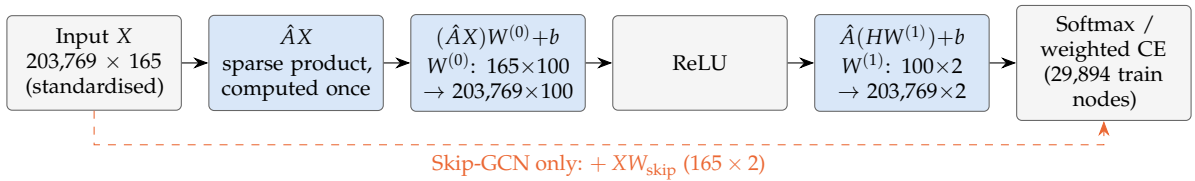


Figure 3: Forward pass of the implemented network with tensor shapes. $\hat{A}$ (sparse, fixed, 672,479 non-zeros) is used in both graph convolutions; only the W s and biases are learned.

The notebook uses plain **PyTorch**. PyTorch Geometric [6] appears only as an optional downloader for the raw CSV files; the graph convolution itself is written from scratch, with no GCNConv or other pre-built layer. $\hat{A}$ is built explicitly from the edge list:

```

def build_norm_adj(src, dst, n):
    loops = torch.arange(n)
    row = torch.cat([src, dst, loops])          # both directions + self-loops -> A~ = A + I
    col = torch.cat([dst, src, loops])
    key = torch.unique(row * n + col)          # remove duplicate edges
  
```

```

row, col = key // n, key % n
d = torch.zeros(n).scatter_add_(0, row, torch.ones(row.numel())) # D~_ii
w = d[row].pow(-0.5) * d[col].pow(-0.5) # D~^-1/2 A~ D~^-1/2
return torch.sparse_coo_tensor(torch.stack([row, col]), w, (n, n)).coalesce()

```

and each layer is one line of linear algebra:

```

class GCNLayer(nn.Module): # H_out = A_hat @ (H W) + b
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.W = nn.Parameter(torch.empty(in_dim, out_dim))
        self.b = nn.Parameter(torch.zeros(out_dim))
        nn.init.xavier_uniform_(self.W)
    def forward(self, A, H, AH=None):
        if AH is not None: # A_hat (H W) = (A_hat H) W
            return AH @ self.W + self.b
        return torch.sparse.mm(A, H @ self.W) + self.b

```

A GCN module stacks these layers with ReLU in between and optionally adds the skip path XW_{skip} ; the MLP is the same module called with the sparse identity matrix in place of $\hat{A}$.

Scale. A dense $203,769 \times 203,769$ matrix would take about 166 GB in float32, whereas $\hat{A}$ has only 672,479 non-zeros ($2 \times 234,355$ edge entries plus 203,769 self-loops), so it is stored as a sparse tensor. Because $\hat{A}$ and X never change and we use no dropout, $\hat{A}(XW^{(0)}) = (\hat{A}X)W^{(0)}$ lets us compute $\hat{A}X$ once before training, which saves one sparse product per epoch in each direction. The GCN has **16,802 trainable parameters** ($165 \times 100 + 100$ in layer 1, $100 \times 2 + 2$ in layer 2), about 0.56 per labelled training transaction, far fewer than on Amazon Photo or Cora, so there is little room to memorise the training set.

Hidden units	100 (as Weber et al.)
Optimiser	Adam, learning rate 0.01, weight decay 5×10^{-4} (Kipf & Welling defaults)
Epochs	200, full batch, last epoch reported
Dropout	none (as Weber et al.)
Loss	weighted cross-entropy, weights 0.3 (licit) / 0.7 (illicit)
Decision threshold	0.5
Seeds	42 for the detailed run; 42, 1, 2, 3, 4 for mean $\pm$ std

Table 2: Hyperparameters, fixed before looking at test results.

Baselines. To judge what the GCN adds, every model is trained on **all features (AF, 165)** and on **local features only (LF, 93)**. The LF setting is the cleaner test of the graph, because the 72 aggregated features are themselves hand-crafted one-hop neighbourhood summaries: with AF, part of what a GCN would learn is already handed to every model. The baselines are Logistic Regression, a Random Forest with 50 trees and `max_features=50` (the configuration of [2]), and our MLP. Finally, following Weber et al., we feed the 100-dimensional hidden layer of the trained GCN, together with the raw features, into a Random Forest (**RF + GCN embedding**).

Depth experiment. To see over-smoothing in a *trained* model, we train GCNs with 1, 2, 3, 4 and 6 layers on local features (2 seeds each) and record the test illicit F1 and the mean pairwise cosine similarity of the last hidden layer on 2,000 random test transactions.

5 Results

5.1 Training run (seed 42, all features)

Epoch	Loss	Train F1	Test F1	Train acc.	Test acc.
1	1.0065	0.479	0.271	80.7%	82.2%
20	0.1961	0.766	0.396	94.1%	90.8%
50	0.1412	0.828	0.417	95.8%	92.6%
100	0.1107	0.871	0.500	96.9%	94.5%
150	0.0928	0.893	0.535	97.4%	95.2%
200	0.0807	0.908	0.572	97.8%	95.5%

Table 3: Selected rows of the training log (F1 = illicit F1; loss measured in training mode, metrics in evaluation mode).

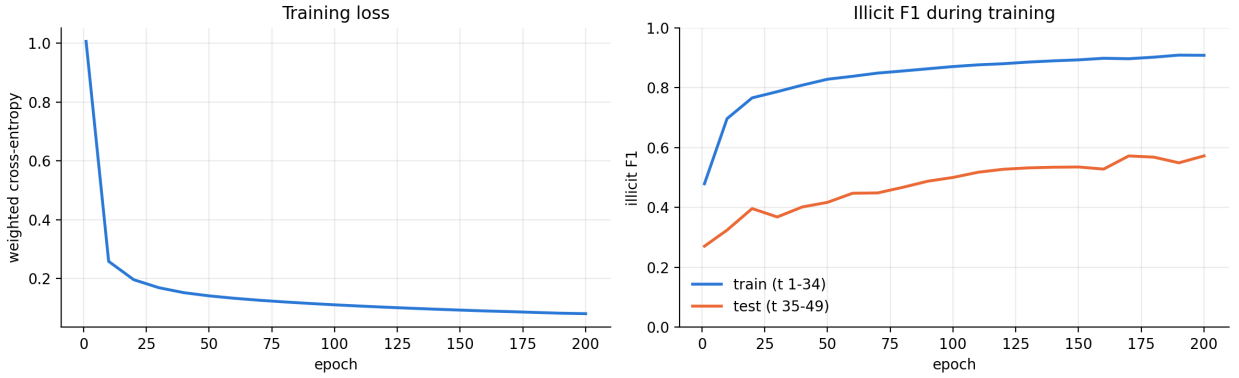


Figure 4: Training loss (left) and illicit F1 on the training and test periods (right). The test curve is plotted only; it is not used to choose the epoch.

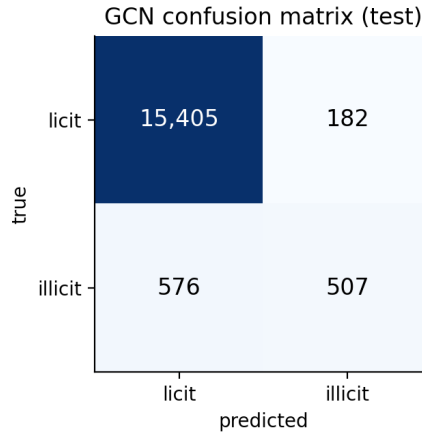


Figure 5: Confusion matrix of the GCN on the 16,670 labelled test transactions (threshold 0.5).

After 200 epochs the GCN flags 689 transactions, of which 507 are illicit (precision 0.736), and it catches 507 of the 1,083 illicit ones (recall 0.468): illicit F1 = **0.572**, AP = 0.593, accuracy 95.5%.

5.2 Comparison with baselines (5 seeds)

Features	Model	Precision	Recall	Illicit F1	AP	Weber et al. F1
AF (165)	Logistic Regression	0.311	0.680	0.427	0.322	0.481
	MLP ($\hat{A} = I$)	0.556	0.539	0.546 ± 0.009	0.443	0.653
	GCN	0.786	0.439	0.560 ± 0.034	0.604	0.628
	Skip-GCN	0.756	0.466	0.575 ± 0.024	0.597	0.705
	Random Forest	0.923	0.722	0.810 ± 0.004	0.783	0.788
	RF + GCN embedding	0.924	0.719	0.809 ± 0.010	0.777	0.796*
LF (93)	Logistic Regression	0.393	0.714	0.507	0.299	0.457
	MLP ($\hat{A} = I$)	0.447	0.745	0.559 ± 0.017	0.519	0.649
	GCN	0.428	0.599	0.495 ± 0.018	0.526	–
	Skip-GCN	0.386	0.619	0.475 ± 0.014	0.493	–
	Random Forest	0.848	0.705	0.770 ± 0.004	0.770	0.694

Table 4: Test performance (time steps 35–49), mean over seeds 42, 1, 2, 3, 4 ($\pm$ std of the illicit F1; Logistic Regression is deterministic). The last column gives the published illicit F1 of Weber et al. [2] for the same split. *Weber et al. used embeddings of a GCN trained with their settings (“AF+NE”).

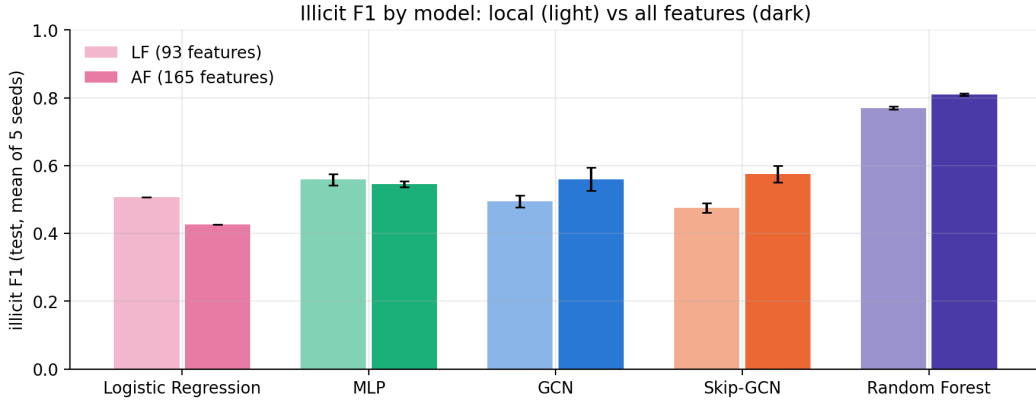


Figure 6: Illicit F1 of every model with local features (light) and all features (dark); error bars are one standard deviation over 5 seeds.

5.3 Performance over time

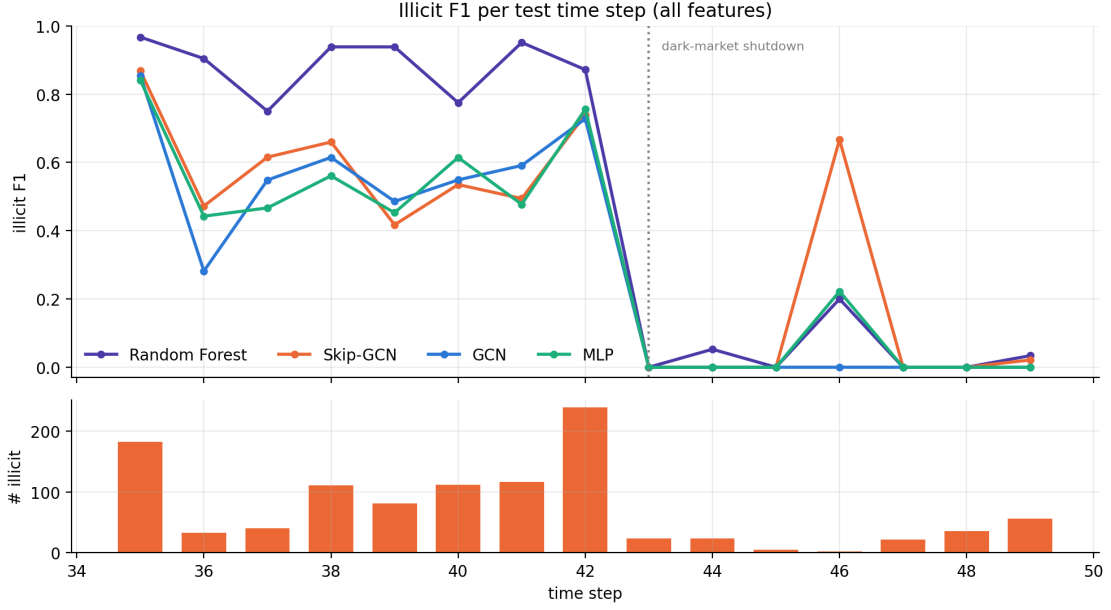


Figure 7: Illicit F1 of each model per test time step (seed 42, all features), with the number of illicit transactions per step below.

5.4 Depth and over-smoothing

Layers	1	2	3	4	6
Illicit F1 (test)	0.273	0.501	0.422	0.385	0.336
std over 2 seeds	0.002	0.021	0.032	0.004	0.022
Cosine sim. of last hidden layer	–	0.410	0.674	0.902	0.849

Table 5: GCNs of increasing depth on local features (seeds 42 and 1). A 1-layer GCN has no hidden layer.

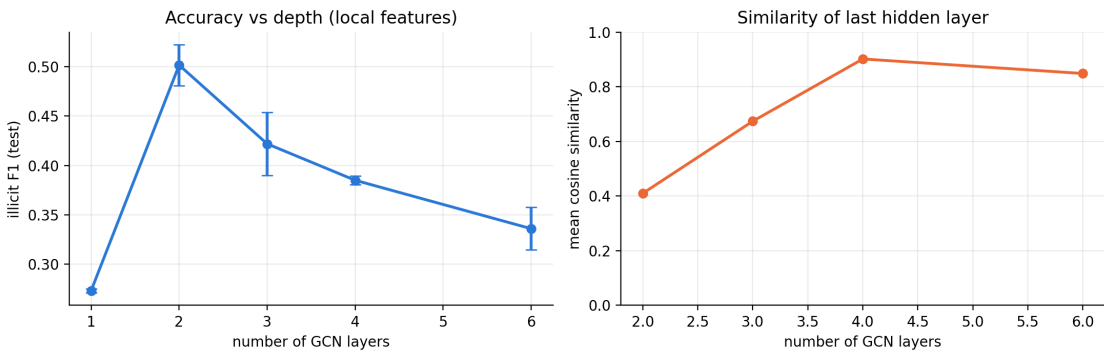


Figure 8: Illicit F1 (left) and mean pairwise cosine similarity of the last hidden layer on 2,000 test transactions (right) versus depth.

6 Analysis and Limitations

6.1 Training behaviour

The first loss is 1.007, above the $\ln 2 \approx 0.693$ of a uniform guess, because the randomly initialised network starts out confidently wrong on some nodes; within 20 epochs it falls to 0.196 and then

decreases smoothly to 0.081 (Fig. 4). The curve has no dropout noise because we use none. Training illicit F1 climbs quickly to about 0.77 and keeps improving slowly to 0.91, while the test F1 rises more gradually from 0.27 to 0.57 and is still increasing at epoch 200. We did not extend training, because choosing the epoch from this curve would amount to tuning on the test set.

6.2 Imbalance and the choice of metric

Test accuracy is 95.5%, only two points above the 93.5% of a model that never flags anything. The illicit F1 of 0.572 is a far more honest picture: the model misses more than half of the illicit transactions. The confusion matrix shows the character of the GCN: when it flags a transaction it is usually right (precision 0.74; 0.79 on average over seeds), but it is conservative (recall 0.44–0.47). Even with the 0.3/0.7 class weights, the effective weight of the illicit class ($0.7 \times 3,462$) is still below that of the licit class ($0.3 \times 26,432$), so the 0.5 threshold sits on the cautious side. A bank would move the threshold according to how many alerts its investigators can handle; the threshold-free AP measures how well the model ranks transactions, independent of that choice.

6.3 What does the graph add?

Comparing the GCN with the identical network that uses $\hat{A} = I$ (the MLP) isolates the graph:

- **With all features**, the graph makes the classifier much more precise: precision rises from 0.556 to 0.786 and AP from 0.443 to 0.604 (+16 points), while recall drops from 0.539 to 0.439. At the fixed 0.5 threshold, illicit F1 improves only slightly ($0.546 \rightarrow 0.560$, within one standard deviation). The Skip-GCN, which keeps a direct path for each transaction’s own features, has the best F1 of the neural models (0.575).
- **With local features only**, the graph does *not* improve F1 (0.559 for the MLP versus 0.495 for the GCN), and AP is about the same (0.519 versus 0.526).

The second result goes against the intuition that the graph should matter most when the hand-made neighbourhood features are removed. It is consistent with the original study, where the MLP (0.653) also beat the GCN (0.628) on F1. We see three plausible reasons, which we did not test separately. (1) *Mixing with unlabelled nodes*: 77% of transactions are unlabelled, and an average degree of 2.3 means that Eq. (3) replaces roughly half to two thirds of a node’s own signal with that of its neighbours, most of which carry no supervision. (2) *Class-level rather than node-level homophily*: 95% of labelled edges are same-class, but this figure is dominated by licit–licit edges. A labelled neighbour of an illicit transaction is illicit only 54% of the time, so averaging can dilute exactly the rare signal we are looking for. (3) *Distribution shift*: test time steps are new graphs, and structural patterns learned on time steps 1–34 transfer less well than per-transaction features (the GCN reaches a training F1 of 0.91 but a test F1 of 0.57). The graph therefore helps a neural model *rank* transactions, and it helps most when combined with features that already describe the neighbourhood, but it is not a free improvement on this dataset.

6.4 Comparison with the published results and the Random Forest

Our Random Forest (0.810 with AF, 0.770 with LF) matches or exceeds Weber et al. (0.788, 0.694), and our GCN (0.560 ± 0.034) and Skip-GCN (0.575) are below their 0.628 and 0.705. The gap is plausibly due to training: they trained for 1,000 epochs at learning rate 0.001, while we used the Kipf–Welling defaults for 200 epochs without tuning, and our test curve was still rising. The Random Forest is clearly the strongest single model. Tree ensembles handle heavy-tailed, heterogeneous tabular features (counts, amounts, fees) without scaling, and they model sharp thresholds and feature interactions that a 100-unit network smooths over. Adding the GCN’s hidden layer to the Random Forest’s inputs did not help (0.809 versus 0.810), unlike the small gain ($0.788 \rightarrow 0.796$) reported by Weber et al. with their better-trained GCN.

6.5 The dark-market shutdown

Figure 7 is perhaps the most important result. Up to time step 42 every model performs reasonably, and the Random Forest is excellent (F1 0.75–0.97). From time step 43, when a large darknet market was shut down, *every* model collapses to an F1 near zero. The spike at $t = 46$ is noise: that time step contains only 2 illicit transactions. The criminals’ behaviour changed, and models trained on the past no longer recognise it. No choice of architecture fixes this: it is **concept drift**, and in practice it calls for continual retraining with fresh labels, drift monitoring, and models that detect *anomalies* rather than only known patterns. Temporal models such as EvolveGCN [3] were proposed for this reason, yet Weber et al. report that they too fail after the shutdown.

6.6 Over-smoothing

Why it happens. $\hat{A}$ is symmetric with eigenvalues in $(-1, 1]$. On a connected component its largest eigenvalue is 1, with eigenvector $\pi \propto \tilde{D}^{1/2}\mathbf{1}$, and all other eigenvalues satisfy $|\mu_i| < 1$ (self-loops rule out -1). Therefore

$$\hat{A}^k H = \pi \pi^\top H + \sum_{i \geq 2} \mu_i^k u_i u_i^\top H \longrightarrow \pi \pi^\top H \quad (k \rightarrow \infty),$$

so every row becomes a multiple of the same vector: all transactions in a time step end up with the same representation direction [4, 5]. $\hat{A}$ is a low-pass filter, and each layer removes a little more of the high-frequency information that distinguishes a node from its neighbours.

What we observe. Table 5 shows this in trained models. A 1-layer GCN is linear and weak (0.273). Two layers are best (0.501). Beyond that, F1 falls steadily (0.422, 0.385, 0.336) while the cosine similarity of the last hidden layer rises from 0.41 to 0.90 at four layers. The value at six layers (0.85) is slightly lower and varies more between seeds (0.09 std), which we attribute to optimisation becoming unstable in a deep network without residual connections or normalisation. Because each Elliptic time step is its own component of up to 7,880 nodes, deep GCNs push illicit and licit transactions of the same time step towards one representation, which is exactly what a fraud detector must avoid. This is why GCNs are used with only two or three layers in practice.

6.7 Limitations

Of our experiment

- Hyperparameters were fixed, not tuned. A validation period (e.g. time steps 30–34) could be used to tune the learning rate, epochs, class weights and decision threshold; our GCN is likely under-trained relative to Weber et al.
- Five seeds (two for the depth study) give rough standard deviations, and differences below about 2 F1 points between neural models are not significant.
- The test period contains only 1,083 illicit transactions, and the post-shutdown steps contain very few, so per-step F1 values after $t = 43$ are noisy.
- The 0.5 threshold is not optimal for F1 for any model; AP is reported to compensate.

Of the vanilla GCN on this problem

- *Fixed, isotropic aggregation:* every neighbour is weighted only by degree. Attention (GAT) or learned neighbour weighting could separate an illicit node’s relevant links from its licit ones.
- *Direction is ignored:* we symmetrise the graph, although money flow is directed; the in- and out-neighbourhoods of a laundering transaction mean different things.
- *Static and transductive:* the model cannot adapt to drift and needs the whole graph in memory (remedies: GraphSAGE-style sampling, temporal GNNs, periodic retraining).

- *Over-smoothing* limits depth (remedies: residual/skip connections, JK-Net, APPNP, DropEdge).

6.8 Conclusion

We implemented a two-layer GCN from scratch, as sparse matrix products $\hat{A}(HW)$, and applied it to a real anti-money-laundering task on 203,769 Bitcoin transactions. On future transactions it reaches an illicit F1 of 0.560 ± 0.034 (0.572 for seed 42). Compared with the same network without the graph, it is far more precise (0.79 versus 0.56) and ranks illicit transactions much better (AP 0.60 versus 0.44). The graph is not a free improvement, though: with local features only it does not beat the MLP, and a Random Forest on tabular features remains the strongest model (0.810), as in the original benchmark. The per-time-step analysis shows the most important lesson of the dataset: after a dark-market shutdown every model fails, so in real AML systems handling concept drift matters more than the choice between a GCN and a tree ensemble. The depth study confirms over-smoothing in trained models and explains why GCNs are kept shallow.

References

- [1] T. N. Kipf and M. Welling. Semi-Supervised Classification with Graph Convolutional Networks. *ICLR*, 2017.
- [2] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson and C. E. Leiserson. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics. *KDD Workshop on Anomaly Detection in Finance*, 2019.
- [3] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. B. Schardl and C. E. Leiserson. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. *AAAI*, 2020.
- [4] Q. Li, Z. Han and X.-M. Wu. Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. *AAAI*, 2018.
- [5] K. Oono and T. Suzuki. Graph Neural Networks Exponentially Lose Expressive Power for Node Classification. *ICLR*, 2020.
- [6] M. Fey and J. E. Lenssen. Fast Graph Representation Learning with PyTorch Geometric. *ICLR Workshop*, 2019.